



РЭУ.РФ
РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ Г.В. ПЛЕХАНОВА

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по учебной практике

УП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции
программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____
(подпись)

Каракулов Николай Витальевич
(фамилия, имя, отчество)

Группы ИПс-21

Руководитель практической подготовки от профильной организации

Преподаватель _____
(должность)

(подпись)

Берестов Дмитрий Борисович
(фамилия, имя, отчество)

МП

«26» июня 2026 года

Руководитель практической подготовки от института

(подпись)

Берестов Дмитрий Борисович
(фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
Раздел 1. Разработка требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент	5
1.1. Сбор информации о существующем состоянии продукта.....	5
Раздел 2. Интеграция модулей в программное обеспечение	12
2.1. Процесс разработки программного обеспечения.....	12
2.2. Разработка программного модуля	13
Раздел 3. Отладка программного модуля с использованием специализированных программных средств	31
3.1. Процесс отладки.....	31
Раздел 4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения	33
4.1. Процесс тестирования.....	33
Раздел 5. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования	37
5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования	37
Заключение.....	41
Список используемых источников.....	43
Требование к написанию программного модуля.....	45

ВВЕДЕНИЕ

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Учебная практика направлена на формирование у обучающихся первоначальных практических профессиональных умений в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем

В настоящей работе представлен отчет о прохождении учебной практики студента-практиканта. Основными видами деятельности являлись: Разработка программного обеспечения; выполнение отладки программного модуля с использованием специализированных программных средств; осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения; уметь производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования

Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Разрабатывать требования к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент;
2. Выполнять интеграцию модулей в программное обеспечение;
3. Выполнять отладку программного модуля с использованием специализированных программных средств;

4. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения.
5. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования
6. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;
7. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности
8. Планировать и реализовывать собственное профессиональное и личностное развитие;
9. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами
10. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.
11. Использовать информационные технологии в профессиональной деятельности
12. Пользоваться профессиональной документацией на государственном и иностранном языках

РАЗДЕЛ 1. РАЗРАБОТКА ТРЕБОВАНИЯ К ПРОГРАММНЫМ МОДУЛЯМ НА ОСНОВЕ АНАЛИЗА ПРОЕКТНОЙ И ТЕХНИЧЕСКОЙ ДОКУМЕНТАЦИИ НА ПРЕДМЕТ ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТ

1.1. Сбор информации о существующем состоянии продукта

Постановка задач:

В рамках учебной практики требуется разработать программный модуль для автоматизации процесса управления персональными задачами. Основные задачи, которые должны быть решены в ходе разработки:

- Обеспечить хранение информации о задачах (название, описание, приоритет, срок выполнения, статус).
- Предоставить пользователю интерфейс для выполнения операций CRUD (создание, чтение, обновление, удаление) над задачами.
- Реализовать механизмы фильтрации и поиска для быстрого доступа к нужным записям.
- Обеспечить сохранность данных при перезапуске приложения с помощью файлового ввода-вывода.
- Разделить бизнес-логику и пользовательский интерфейс (архитектура с динамической библиотекой классов и WPF-клиентом).
- Обеспечить надёжность работы через модульное тестирование ключевых компонентов.
- Предоставить дополнительные возможности для удобства (сортировка, маркировка важных задач, простая статистика).

Описание бизнес-процесса (состояние «AS IS» – «как есть»):

На текущий момент (до внедрения системы) управление личными задачами выполняется вручную с использованием неформализованных методов:

- Пользователь записывает задачи на бумажных стикерах или в текстовом файле без единой структуры.

- Отсутствует классификация по статусам и приоритетам; информация хранится в произвольном виде.
- Поиск нужной задачи осуществляется визуальным просмотром всего списка, что занимает время и приводит к ошибкам.
- Контроль сроков выполнения не автоматизирован – пользователь полагается на собственную память.
- Данные не структурированы, нет возможности получить агрегированную информацию (статистику).
- При потере файла или записи данные восстанавливаются вручную с большими трудозатратами.

Перечень действий в состоянии «AS IS»:

1. Пользователь формулирует задачу в уме или кратко записывает в блокнот.
2. При необходимости вспомнить задачу – просматривает все записи подряд.
3. Для отметки выполнения – ищет запись и помечает её "галочкой" (в лучшем случае) или удаляет.
4. Поиск просроченных задач – ручной перебор и сравнение с текущей датой.
5. Ведение списка – хаотичное добавление новых записей без систематизации.

Недостатки процесса:

- Низкая скорость работы.
- Отсутствие единого хранилища.
- Невозможность быстрой фильтрации и сортировки.
- Высокий риск потери данных.
- Отсутствие контроля над сроками.

Описание процесса модернизации (состояние «TO BE» – «как будет»):

После внедрения разрабатываемого приложения процесс управления задачами автоматизируется:

- Все задачи хранятся в структурированном виде (объекты класса Task) с четко заданными атрибутами.
- Данные сохраняются в файл (JSON/XML), что обеспечивает постоянное хранение и загрузку при старте программы.
- Пользователь через WPF-интерфейс может добавлять, редактировать, удалять задачи, а также выполнять поиск и фильтрацию в реальном времени.
- Реализована возможность сортировки списка по приоритету или сроку, что упрощает планирование.
- Визуальное выделение важных задач (звёздочка или цвет) помогает фокусироваться на критичных пунктах.
- Автоматический подсчет статистики (количество завершённых, просроченных задач) даёт мгновенную оценку эффективности.
- Бизнес-логика изолирована в библиотеке классов, что позволяет тестировать её независимо от интерфейса.

Перечень действий в состоянии «ТО ВЕ»:

1. Пользователь запускает WPF-приложение.
2. При загрузке данные автоматически подгружаются из файла (если он существует).
3. Для добавления задачи – заполняет форму в интерфейсе и нажимает "Сохранить"; задача добавляется в коллекцию и сохраняется в файл.
4. Для просмотра списка – открывает главное окно, где все задачи отображаются в табличной форме.
5. Для фильтрации – выбирает статус из выпадающего списка, список мгновенно обновляется.
6. Для поиска – вводит ключевое слово в строку поиска, отображаются только подходящие задачи.

7. Для редактирования – выбирает задачу, изменяет поля и сохраняет изменения.

8. Для удаления – выбирает задачу и нажимает кнопку "Удалить" (с подтверждением).

9. При завершении работы приложения данные автоматически сохраняются в файл.

10. Дополнительно – может включить сортировку, посмотреть статистику или отметить задачу как "Важную".

1.2. Разработка технического задания

Основные требования к техническому заданию:

Техническое задание (ТЗ) должно содержать:

1. Полное наименование и назначение разрабатываемого программного продукта.

2. Перечень требований к функциональным характеристикам (что должно делать ПО).

3. Требования к составу и параметрам технических средств (минимальные системные требования).

4. Требования к программной совместимости (ОС, платформа .NET, форматы данных).

5. Требования к надёжности, эргономике и интерфейсу пользователя.

6. Требования к документированию и поставке.

7. Этапы разработки и контрольные точки.

8. Требования к тестированию (в том числе модульному).

В учебном проекте ТЗ разработано на основе технического задания, выданного преподавателем, и дополнено архитектурными решениями.

Стандарты, используемые для написания технического задания

Разработка ТЗ выполняется в соответствии с требованиями следующих стандартов:

- **ГОСТ 19.201-78 (ЕСПД)** – Техническое задание. Требования к содержанию и оформлению (для разработки программной документации).

- **ГОСТ 34.602-89** – Техническое задание на создание автоматизированной системы (для системного уровня, применяется в укрупненном виде для описания процесса автоматизации).

- Также учитываются рекомендации по оформлению текстовых документов согласно ГОСТ 2.105-95 (ЕСКД).

Содержание технического задания

На основе перечисленных стандартов и выданного задания разработано следующее содержание ТЗ:

1. Введение

- Наименование системы: «Task Manager WPF».
- Цель создания: автоматизация управления персональными задачами.

2. Основания для разработки

- Учебная практика по дисциплине «Программирование на C#».

3. Назначение разработки

- Для конечного пользователя (физическое лицо) – ведение списка задач с возможностью фильтрации, поиска, редактирования, сохранения данных.

4. Требования к программе

- **Функциональные требования:** (п. 1.1–1.8 и 2.1–2.3 из исходного задания).

- **Требования к надёжности:** модульные тесты для ключевых классов; обработка исключений при работе с файлами.

- **Условия эксплуатации:** Windows 10/11, установленный .NET Runtime 6.0 или выше.

- **Требования к составу и параметрам технических средств:** ПК с тактовой частотой процессора не менее 1.5 ГГц, ОЗУ от 1 ГБ.

- **Требования к информационной и программной совместимости:** платформа .NET, файлы JSON/XML.

- **Требования к интерфейсу:** WPF с использованием паттерна MVVM (Model-View-ViewModel).

- **Требования к документированию:** сопроводительная записка с иллюстрациями, диаграмма классов, описание действий по созданию ПО.

5. Требования к программной документации

– В состав документации входит техническое задание, пояснительная записка, руководство пользователя (краткое).

6. Техничко-экономические показатели

– Не рассчитываются (учебный проект).

7. Этапы разработки

– Анализ требований → проектирование (диаграмма классов) → разработка библиотеки классов → модульное тестирование → разработка WPF-интерфейса → интеграция → тестирование системы → оформление документации.

8. Порядок контроля и приёмки

– Проверка выполнения всех функциональных требований, прохождение модульных тестов, соответствие интерфейса требованиям.

Структура технического задания

В соответствии с ГОСТ 19.201-78, структура ТЗ включает следующие разделы (адаптировано для проекта):

1. Титульный лист (наименование организации, название работы, подписи).
2. Содержание (оглавление разделов).
3. Введение (цель, назначение).
4. Основание для разработки.
5. Назначение разработки.
6. Требования к программе:
7. Требования к функциональным характеристикам.
8. Требования к надёжности.
9. Условия эксплуатации.

10. Требования к составу и параметрам технических средств.
11. Требования к информационной и программной совместимости.
12. Требования к интерфейсу пользователя.
13. Требования к программной документации.
14. Техничко-экономические показатели.
15. Стадии и этапы разработки.
16. Порядок контроля и приёмки.
17. Приложения (схемы, диаграммы, протоколы согласования).

Порядок документирования требований

Документирование требований осуществляется в следующем порядке:

1. **Сбор требований** – на основе исходного технического задания, дополнительных уточнений и анализа предметной области.
2. **Формализация требований** – запись требований в виде структурированного документа (ТЗ) с использованием терминологии предметной области.
3. **Согласование** – проверка ТЗ с заказчиком (в учебном проекте – с преподавателем) и утверждение.
4. **Ведение изменений** – при необходимости вносятся изменения в ТЗ с фиксацией в листе регистрации изменений.
5. **Использование в разработке** – ТЗ служит основой для проектирования, кодирования, тестирования и оценки результата.
6. **Контроль соответствия** – на всех этапах разработки проверяется выполнение требований, зафиксированных в ТЗ.

РАЗДЕЛ 2. ИНТЕГРАЦИЯ МОДУЛЕЙ В ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1. Процесс разработки программного обеспечения

Выбор модели разработки и обоснование:

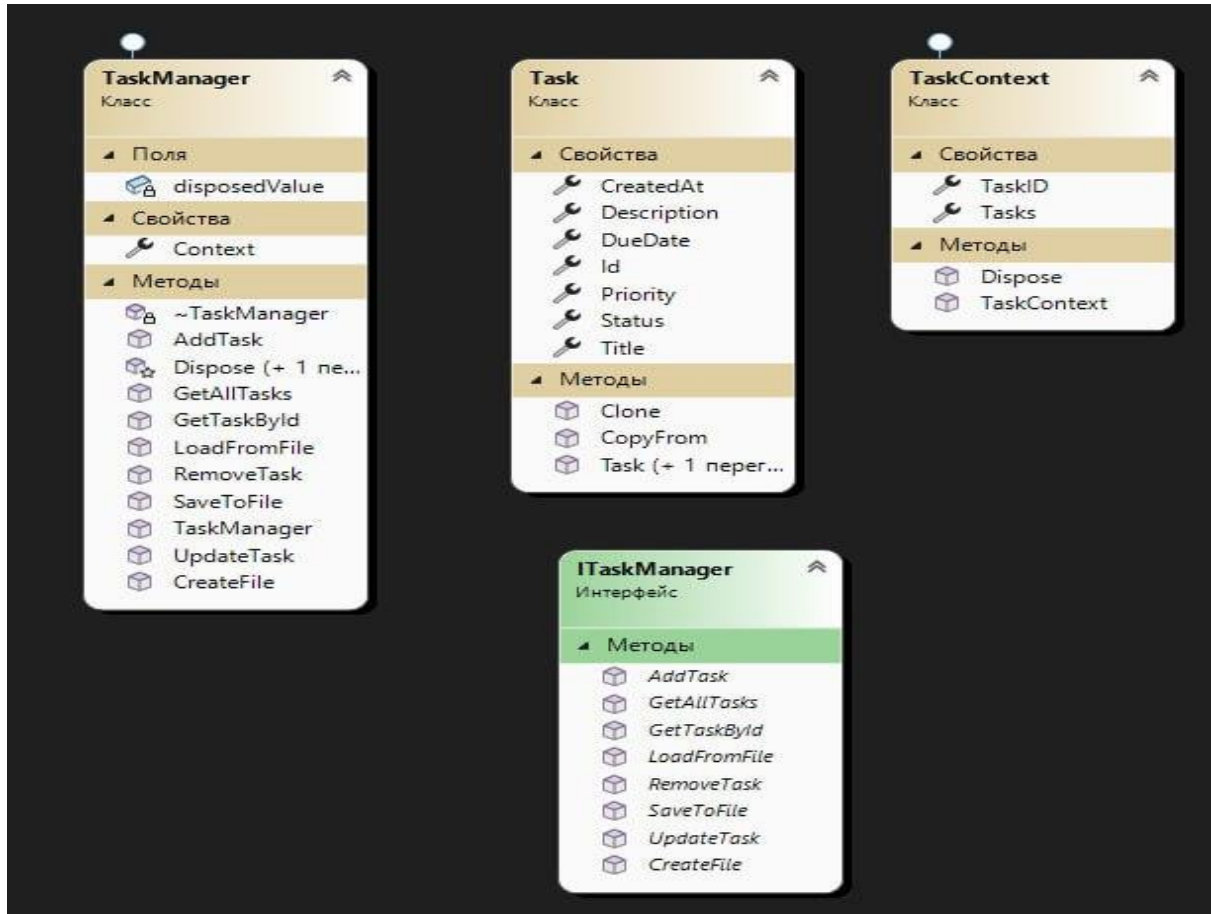
В ходе создания приложения «KarakulovLib» была выбрана **каскадная (водопадная) модель** жизненного цикла программного обеспечения. Данное решение обосновано следующими факторами:

1. **Чётко определённые требования** – техническое задание было выдано в полном объёме на старте проекта, содержало все функциональные и нефункциональные требования, а также перечень дополнительных возможностей. Изменения требований в процессе разработки не предполагались, что соответствует классическому водопадному подходу.
2. **Небольшой размер и относительная простота проекта** – приложение состоит из трёх проектов (WPF-интерфейс, библиотека классов, модульные тесты) и реализует стандартный набор операций CRUD с сохранением в файл. Такой объём работ удобно планировать и выполнять последовательно, без необходимости частых возвратов на предыдущие этапы.
3. **Жёсткая последовательность этапов** – разработка велась поэтапно: анализ требований → проектирование архитектуры → написание библиотеки классов → модульное тестирование → разработка пользовательского интерфейса → интеграция и системное тестирование → оформление документации. Каждый этап завершался перед переходом к следующему, что позволяло контролировать качество промежуточных результатов.
4. **Акцент на документирование** – поскольку проект выполняется в учебных целях, большое внимание уделяется созданию сопроводительной документации (пояснительная записка, диаграмма

классов, иллюстрации). Водопадная модель обеспечивает строгую фиксацию всех артефактов на каждом этапе.

5. **Прогнозируемость сроков** – при фиксированных требованиях и небольшом объеме работ каскадная модель позволяет достаточно точно оценить временные затраты и последовательность действий.

Диаграмма классов:



(Рис.1 – Диграмма классов)

2.2. Разработка программного модуля

Реализация ITaskManager.cs

```
namespace KarakulovLib
{
    public interface ITaskManager
    {
        void AddTask(Task task);
        void UpdateTask(Task task);
        void RemoveTask(Task task);
    }
}
```

```

        Task GetTaskById(int id);
        IEnumerable<Task> GetAllTasks();

        void SaveToFile(string filePath);
        void LoadFromFile(string filePath);
        void CreateFile(string filePath);

    }
}

```

Реализация Task.cs

```

namespace KarakulovLib
{
    public class Task
    {
        public int Id { get; set; }

        public string Title { get; set; }

        public string Description { get; set; }

        public string Priority { get; set; }

        public DateTime? DueDate { get; set; }

        public string Status { get; set; }
    }
}

```

```
public DateTime CreatedAt { get; set; }
```

```
public Task()
```

```
{
```

```
    CreatedAt = DateTime.Now;
```

```
}
```

```
public Task Clone()
```

```
{
```

```
    return new Task
```

```
{
```

```
        Id = this.Id,
```

```
        Title = this.Title,
```

```
        Description = this.Description,
```

```
        Priority = this.Priority,
```

```
        DueDate = this.DueDate,
```

```
        Status = this.Status,
```

```
        CreatedAt = this.CreatedAt
```

```
    };
```

```
}
```

```
public void CopyFrom(Task source)
```

```

    {
        this.Title = source.Title;

        this.Description = source.Description;

        this.Priority = source.Priority;

        this.DueDate = source.DueDate;

        this.Status = source.Status;
    }

    public Task(string title, string description, string priority, string
status, DateTime? dueDate)
    {
        Title = title;

        Description = description;

        Priority = priority;

        DueDate = dueDate;

        Status = status;

        CreatedAt = DateTime.Now;
    }
}

```



```

1  using System;
2  using System.Collections.ObjectModel;
3
4  namespace KarakulovLib
5  {
6      [Serializable]
7      public class TaskContext : IDisposable
8      {
9          public ObservableCollection<Task> Tasks { get; set; } = new ObservableCollection<Task>();
10         public static int TaskID { get; set; } = 0;
11
12         public TaskContext()
13         {
14         }
15
16         public void Dispose()
17         {
18             Tasks.Clear();
19             GC.SuppressFinalize(this);
20         }
21     }
22 }

```

(Рис.2 – Реализация TaskContext.cs)

Реализация TaskManager.cs

```

namespace KarakulovLib

{

    public class TaskManager : IDisposable, ITaskManager

    {

        public TaskContext Context { get; private set; } = new
TaskContext();

        private bool disposedValue;

        public TaskManager()

        {

        }

        public void AddTask(Task task)

        {

```

```

        if (!Context.Tasks.Any(t => t.Id == task.Id))
        {
            task.Id = ++TaskContext.TaskID;

            Context.Tasks.Add(task);
        }
    }

    public void UpdateTask(Task task)
    {
        var target = Context.Tasks.FirstOrDefault(t => t.Id ==
task.Id);

        if (target != null)
        {
            target.Title = task.Title;

            target.Description = task.Description;

            target.Priority = task.Priority;

            target.DueDate = task.DueDate;

            target.Status = task.Status;
        }
        else
        {

```

```

        throw new ArgumentException("Задача для обновления
не найдена.");
    }
}

```

```

public void RemoveTask(Task task)
{
    if (Context.Tasks.Contains(task))
    {
        Context.Tasks.Remove(task);
    }
}

```

```

public Task GetTaskById(int id)
{
    return Context.Tasks.FirstOrDefault(t => t.Id == id);
}

```

```

public IEnumerable<Task> GetAllTasks()
{
    return Context.Tasks;
}

```

```

public void SaveToFile(string filePath)
{
    try
    {
        using (FileStream stream = new FileStream(filePath,
FileMode.Create))
        {
            DataContractJsonSerializer serializer = new DataCon-
tractJsonSerializer(typeof(List<Task>));

            serializer.WriteObject(stream, Context.Tasks.ToList());
        }
    }
    catch (Exception ex)
    {
        throw new IOException($"Ошибка при сохранении
задач: {ex.Message}", ex);
    }
}

```

```

public void CreateFile(string filePath)

```

```

    {
        try
        {
            using (FileStream stream = new FileStream(filePath,
FileMode.Create))

            {
                DataContractJsonSerializer serializer = new DataCon-
tractJsonSerializer(typeof(List<Task>));

                serializer.WriteObject(stream, new List<Task>());
            }
        }

        catch (Exception ex)
        {
            throw new IOException($"Ошибка при сохранении
задач: {ex.Message}", ex);
        }
    }

    public void LoadFromFile(string filePath)
    {
        try
        {

```

```

        if (File.Exists(filePath))
        {
            using (FileStream stream = new FileStream(filePath,
FileMode.Open))
            {
                DataContractJsonSerializer serializer = new DataCon-
tractJsonSerializer(typeof(List<Task>));

                var tasks = serializer.ReadObject(stream) as
List<Task>;

                if (tasks != null)
                {
                    Context.Tasks.Clear();

                    foreach (var task in tasks)
                    {
                        Context.Tasks.Add(task);
                    }

                    if (tasks.Any())
                    {
                        TaskContext.TaskID = tasks.Max(t => t.Id);
                    }
                }
            }
        }
    }
}

```

```

        }

    }

}

catch (Exception ex)

{

    throw new IOException($"Ошибка при загрузке задач:
{ex.Message}", ex);

}

}

```

```

protected virtual void Dispose(bool disposing)

{

    if (!disposedValue)

    {

        if (disposing)

        {

            if (Context != null)

            {

                Context.Dispose();

            }

        }

    }

}

```

```

        disposedValue = true;
    }
}

public void Dispose()
{
    Dispose(disposing: true);
    GC.SuppressFinalize(this);
}

~TaskManager()
{
    Dispose(disposing: false);
}
}
}

```

Этапы	разработки	ПО:
На этапе проектирования был определён архитектурный шаблон, обеспечивающий разделение ответственности и упрощающий сопровождение. Для данного приложения выбран паттерн Model-View-ViewModel (MVVM), который является стандартным для WPF-приложений.		

Документирование системы

Документирование выполнялось параллельно с разработкой и включало создание следующих артефактов:

1. **Техническое задание (ТЗ)** – разработано на начальном этапе на основе выданного задания, содержит все функциональные и нефункциональные требования, структуру и этапы разработки.
2. **Пояснительная записка** – итоговый документ, в который вошли:
 - титульный лист;
 - оглавление;
 - диаграмма классов;
 - подробное описание процесса создания ПО с иллюстрациями (скриншоты интерфейса, фрагменты кода, результаты тестов);
 - описание состояний «AS IS» и «TO BE», обоснование выбора модели разработки и инструментов.
3. **Руководство пользователя** (краткое) – описание того, как установить приложение, запустить, выполнять основные операции (добавление, редактирование, удаление, поиск, фильтрация, просмотр статистики).
4. **Комментарии в коде** – для всех публичных методов и классов библиотеки добавлены XML-комментарии (для генерации документации и подсказок в IDE).
5. **Схема базы данных** (в упрощённом виде) – описание формата JSON-файла и структуры задачи, включено в пояснительную записку.

Форма для ввода и редактирования данных:

КаракуловWpf

Название:

Описание:

Приоритет:

Срок выполнения: 24.06.2026

Статус:

OK Отмена

(Рис.3 – Форма для ввода и редактирования данных)

КаракуловWpf

Файл Задачи Фильтры

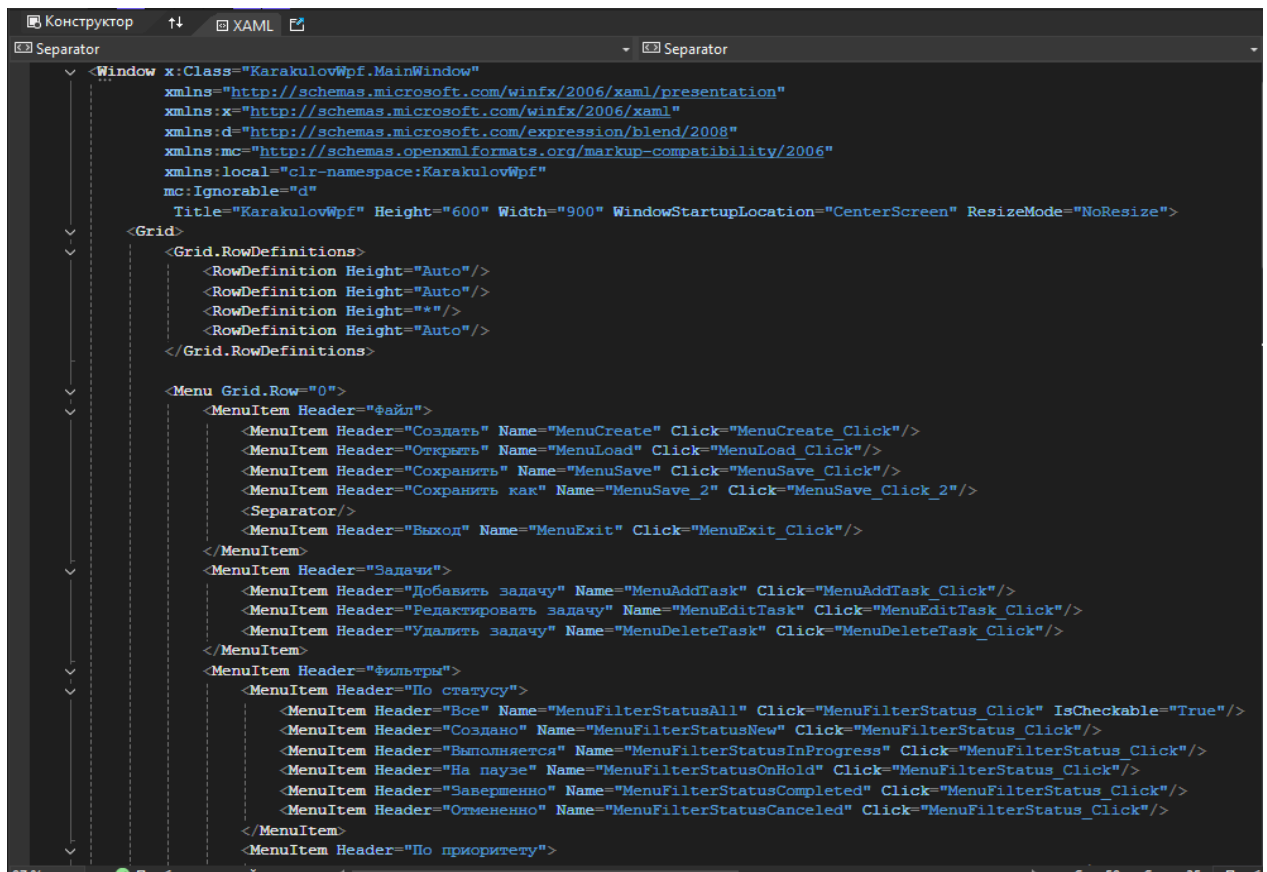
Поиск: Найти Сбросить

ID	Название	Статус	Приоритет	Срок выполнения	Дата создания
1	Взлом пентагона	Выполняется	Критический	24.06.2026	24.06.2026

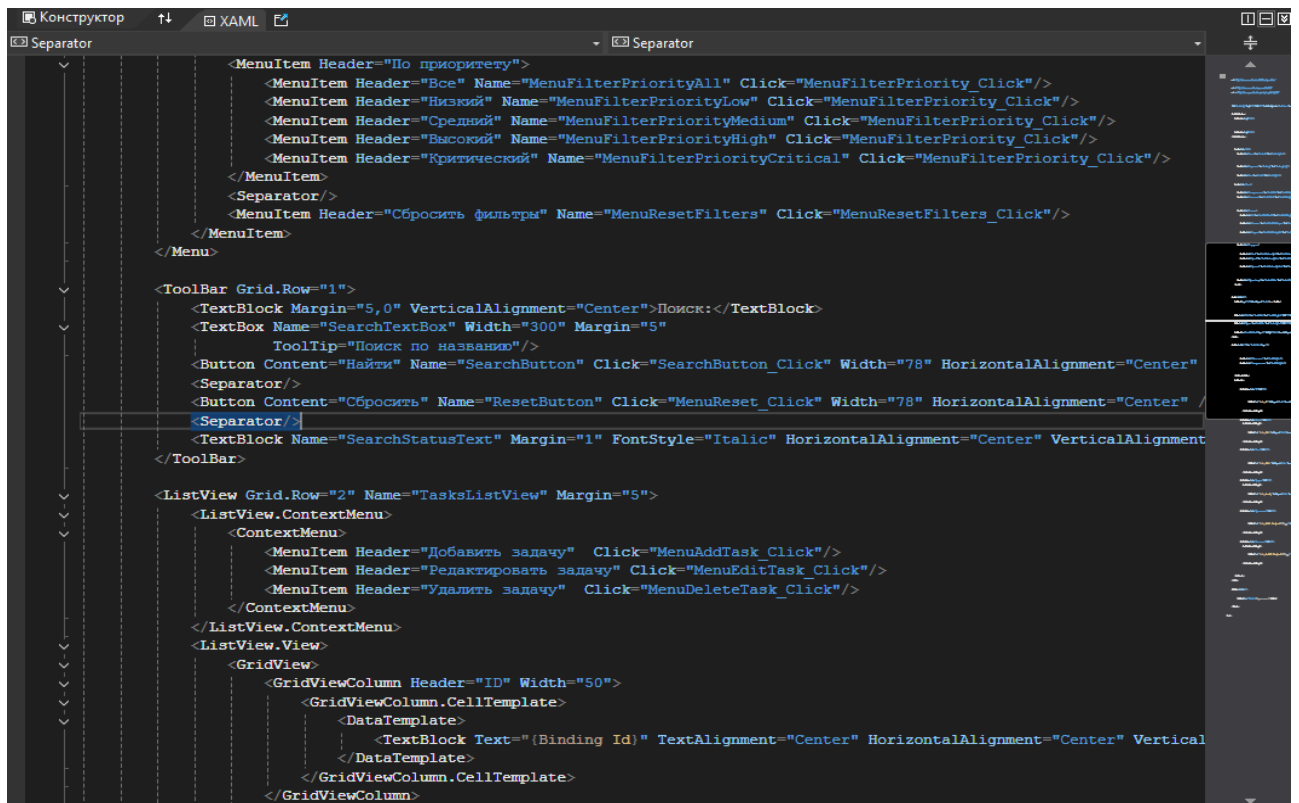
Задач: 1 | Добавлена задача #1

(Рис.4 – Главное окно с возможностью добавления и редактирования задач)

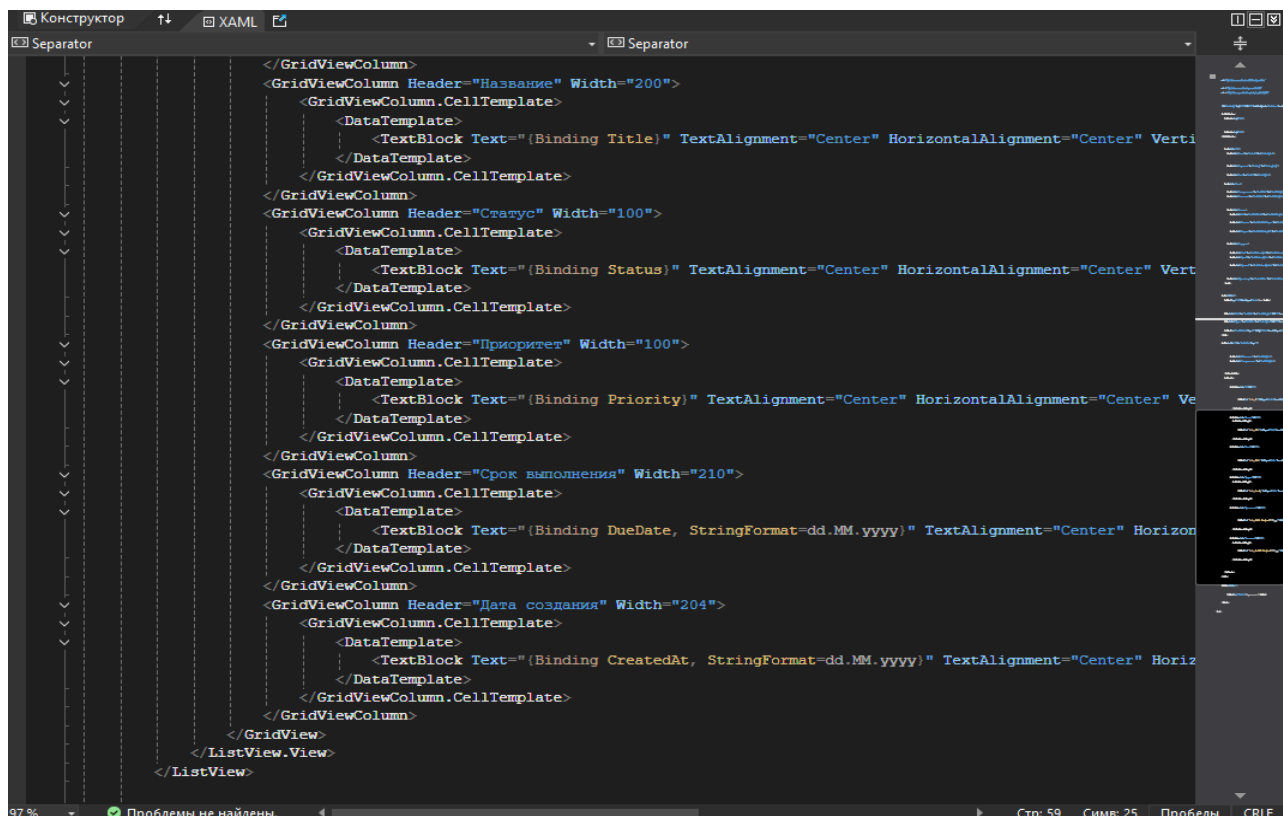
Разработка интерфейса:



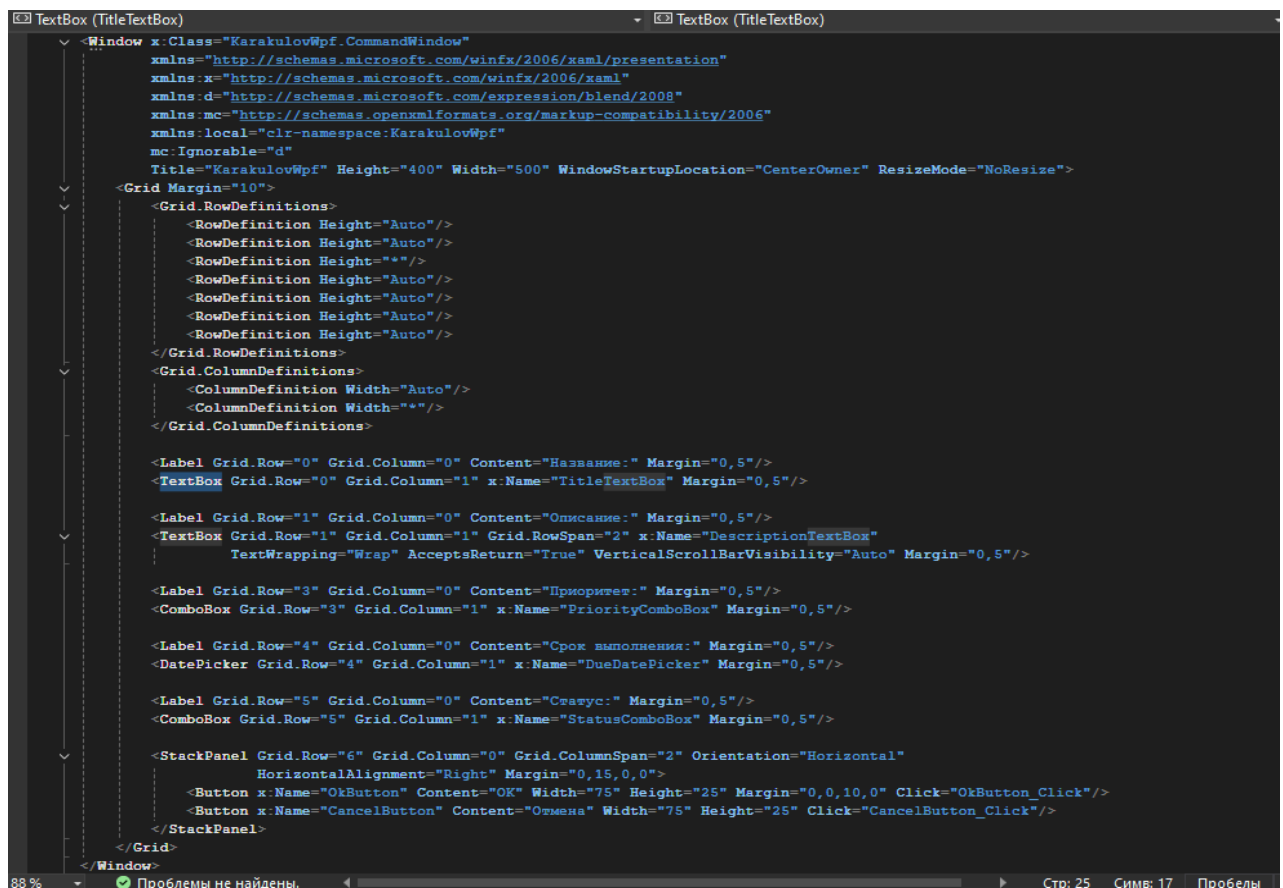
(Рис.5 – Окно MainWindow)



(Рис.6 – Окно MainWindow)



(Рис.7 – Окно MainWindow)



(Рис.8 – Окно CommandWindow)

Описание реализации функционала окна MainWindow.xaml.cs

При старте приложения выполняется следующая логика:

- 1) Создается экземпляр `_taskManager`, управляющий задачами.
- 2) Настраивается представление списка задач (`TasksListView`) через привязку к источнику данных (`CollectionViewSource`).
- 3) Устанавливаются исходные настройки фильтров: изначально выбираются все возможные статусы и приоритеты.

При закрытии окна освобождаются ресурсы посредством метода `_taskManager.Dispose()`.

Для фильтрации и поиска выполняются такие действия:

- 1) Поисковая строка передает запрос на фильтрацию задач.
- 2) Фильтры по статусу и приоритету задают условия отбора задач и применяются автоматически при изменении значений фильтров.

Операции над задачами включают:

- 1) Добавление новой задачи: Пользователь вводит данные в окне `CommandWindow`. Затем новая задача добавляется методом `_taskManager.AddTask`.

- 2) Редактирование существующей задачи: Выбранная задача передается в окно `CommandWindow` для редактирования, после чего изменяется через `_taskManager.UpdateTask`.

- 3) Удаление задачи: Подтверждение удаления осуществляется окном предупреждения, затем задача удаляется методом `_taskManager.RemoveTask`.

Работа с файлами организована следующим образом:

- 1) Создание нового файла: Диалог сохранения предлагает выбрать путь для нового файла, затем используется метод `_taskManager.CreateFile`.
- 2) Загрузка файла: Через диалог открытия файла вызывается метод `_taskManager.LoadFromFile`, загружающий ранее сохраненные задачи.

3) Сохранение текущего набора задач: Используется метод `_taskManager.SaveToFile`, который записывает изменения в существующий файл. Если файл не указан, выдается уведомление об ошибке.

4) Операция "Сохранить как": Открывается диалог выбора пути, после чего задачи сохраняются новым файлом.

Также предусмотрены дополнительные возможности:

1) Возможность сброса всех примененных фильтров и очистки строки поиска для полного восстановления первоначального представления задач.

2) Обновление статусной строки интерфейса, показывающей общее число задач и дополнительную информацию о выполненных операциях (например, "Файл успешно сохранён").

Описание функциональности окна `CommandWindow.xaml.cs`

Это окно предназначено для добавления новых задач либо редактирования существующих.

Основные шаги работы окна:

1) **Инициализация:** Базовый конструктор создаёт новую задачу, второй — принимает задачу для редактирования и копирует её во временный объект, предотвращая прямое изменение исходника.

2) **Элементы интерфейса:** Выпадающие списки приоритета и статуса наполняются стандартными значениями («Критический», «Высокий», «Средний», «Низкий») и («Создано», «Выполняется», «На паузе», «Завершено», «Отменено»), соответственно. Поле срока выполнения устанавливается текущей датой.

Обработка кнопок:

1) Кнопка «ОК»: Проверяются введённые данные, включая наличие имени задачи и корректность даты выполнения. Если проверка пройдена, данные сохраняются в задаче, оригинальная задача обновляется, и окно закрывается с результатом `True`.

2) Кнопка «Отмена»: Окно закрывается без сохранения внесённых изменений.

Проверка и сохранение данных: Метод проверки валидности (ValidateInput) контролирует правильность введённого названия и актуальность даты завершения. Данные задачи сохраняются в соответствующую структуру модели через метод SaveTaskData.

Таким образом, оба окна обеспечивают полный цикл операций по управлению задачами: создание, редактирование, удаление, загрузку/сохранение файлов и работу с интерфейсными элементами для удобства пользователя.

РАЗДЕЛ 3. ОТЛАДКА ПРОГРАММНОГО МОДУЛЯ С ИСПОЛЬЗОВАНИЕМ СПЕЦИАЛИЗИРОВАННЫХ ПРОГРАММНЫХ СРЕДСТВ

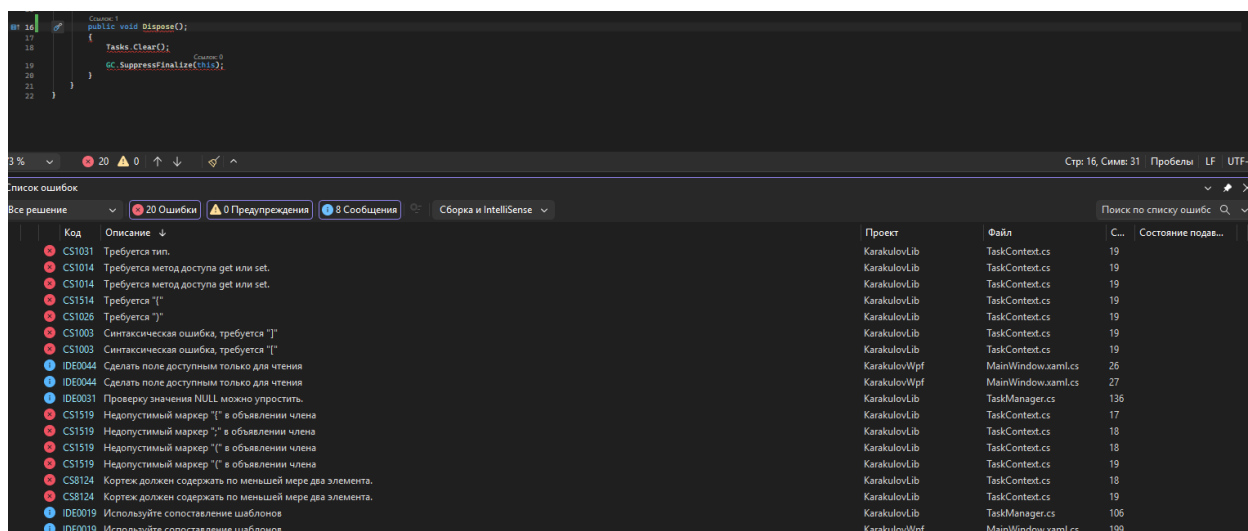
3.1. Процесс отладки

Отладка программного обеспечения является неотъемлемой частью процесса разработки, направленной на выявление, локализацию и устранение ошибок (дефектов) в коде. В ходе создания приложения «Task Manager WPF» отладка проводилась на всех этапах: от написания отдельных методов библиотеки классов до интеграции с пользовательским интерфейсом. Основной целью было обеспечение корректной работы всех функций в соответствии с техническим заданием и достижение высокой стабильности приложения.

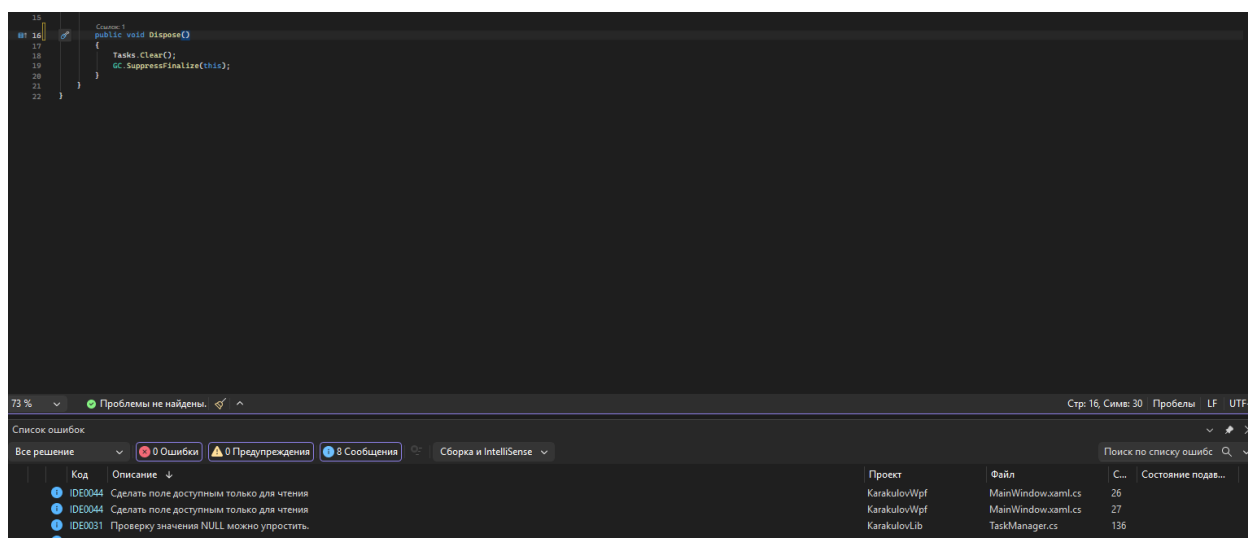
Для отладки использовались следующие инструменты, встроенные в среду разработки **Microsoft Visual Studio Community 2022**:

- **Отладчик (Debugger)** – основной инструмент, позволяющий запускать приложение в режиме отладки, устанавливать точки останова, выполнять код по шагам, анализировать состояние переменных и потоков выполнения.
- **Точки останова (Breakpoints)** – устанавливались как в коде библиотеки, так и в WPF-представлении (code-behind и ViewModel) для приостановки выполнения в ключевых точках.
- **Окна отладки:**
 - «Локальные» (Locals) – для просмотра значений всех локальных переменных в текущем контексте.
 - «Авто» (Autos) – для отображения переменных, используемых в текущей и предыдущей строках.
 - «Контрольные значения» (Watch) – для отслеживания значений конкретных выражений или свойств.

- «Исключения» (Exception Settings) – для настройки остановки при возникновении определённых исключений (включая пользовательские).
- **Окно «Вывод» (Output)** и метод `Debug.WriteLine()` – использовались для вывода отладочных сообщений в консоль отладчика без остановки выполнения, что помогало трассировать поток данных.
- **Средства диагностики (Diagnostic Tools)** – для анализа использования памяти, времени выполнения и событий, что было полезно при проверке производительности.



(Рис.9 – Пример вывода ошибок в консоль)



(Рис.10 – Исправленный код)

РАЗДЕЛ 4. ОСУЩЕСТВЛЯТЬ РАЗРАБОТКУ ТЕСТОВЫХ НАБОРОВ И ТЕСТОВЫХ СЦЕНАРИЕВ ДЛЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

4.1. Процесс тестирования

Описание процесса проведения тестирования ПО

Тестирование программного обеспечения является важнейшим этапом жизненного цикла, обеспечивающим выявление дефектов и подтверждение соответствия продукта функциональным требованиям. В ходе разработки приложения «Task Manager WPF» процесс тестирования был организован на нескольких уровнях:

1. **Модульное тестирование (Unit Testing)** – проверка отдельных компонентов (классов и методов) в изоляции от внешних зависимостей. Для этого использовался фреймворк **MSTest** (Microsoft.VisualStudio.TestTools.UnitTesting), встроенный в Visual Studio. Модульные тесты охватывают основную бизнес-логику, реализованную в библиотеке классов KarakulovLib.

2. **Интеграционное тестирование** – проверка взаимодействия между библиотекой и файловой системой (сохранение/загрузка данных), а также между ViewModel и Model в WPF-приложении. Выполнялось вручную с использованием отладчика и автоматизированных тестов (например, тест TestSaveAndLoadFromFile покрывает интеграцию с JSON-файлом).

3. **Системное тестирование** – проверка работы приложения в целом, включая пользовательский интерфейс, производительность, обработку ошибок и соответствие требованиям технического задания. Проводилось вручную на различных сценариях (добавление, редактирование, удаление, фильтрация, поиск, сортировка, статистика).

4. **Тестирование регрессии** – после каждого исправления ошибок или добавления новой функциональности запускались все модульные тесты, чтобы убедиться, что существующая функциональность не нарушена.

Процесс тестирования включал следующие этапы:

- **Планирование тестирования** – на основе требований ТЗ составлен перечень проверяемых функций.
- **Разработка тестов** – созданы модульные тесты для каждого критического метода библиотеки.
- **Выполнение тестов** – тесты запускались как в среде разработки (через Test Explorer), так и в автоматическом режиме при каждой сборке (при необходимости).
- **Анализ результатов** – выявленные ошибки фиксировались, исправлялись, и тесты запускались повторно до полного прохождения.
- **Оценка покрытия** – вручную оценивалось, какие сценарии покрыты тестами; в перспективе планируется использование средств анализа покрытия кода.

Для изолированного тестирования использовалась **фиктивная файловая система** – тесты используют отдельный тестовый файл `test_data.json`, который создаётся и удаляется в процессе выполнения (методы `[TestInitialize]` и `[TestCleanup]`). Это позволяет избежать влияния на реальные данные пользователя.

Создание unit-тестов

Для тестирования бизнес-логики библиотеки `KarakulovLib` был создан отдельный проект модульных тестов `KarakulovTests` с использованием **MSTest**. Тесты покрывают следующие функциональные области:

№	Тестовый метод	Проверяемый сценарий	Ожидаемый результат
1	<code>TestAddTask_ShouldAddTaskToList</code>	Добавление новой задачи в коллекцию	Задача добавляется, количество задач становится равным 1, название соответствует заданному.
2	<code>TestFindTask_ShouldFindTask</code>	Поиск задачи по идентификатору	Возвращается задача с правильным заголовком.
3	<code>TestGetAllTasks_ShouldGetAllTasks</code>	Получение списка всех задач	Возвращается коллекция, содержащая все добавленные задачи (проверка количества и порядка).
4	<code>TestUpdateTask_ShouldUpdateExistingTask</code>	Обновление существующей задачи (изменение полей)	После обновления задача в коллекции имеет новые значения заголовка и описания.
5	<code>TestRemoveTask_ShouldRemoveTaskFromList</code>	Удаление задачи из коллекции	Задача удаляется, количество задач становится равным 0.
6	<code>TestSaveAndLoadFromFile_ShouldPersistTasks</code>	Сохранение списка задач в JSON-файл и последующая загрузка	После загрузки из файла список задач полностью соответствует сохранённому (по количеству и содержанию).

(Рис.11 – Unit тесты)

```

10  public class MarakulovTests
11  {
12      private TaskManager taskManager;
13      private string testDataPath = "test_data.json";
14
15      [TestInitialize]
16      public void Setup()
17      {
18          taskManager = new TaskManager();
19
20          if (File.Exists(testDataPath))
21          {
22              File.Delete(testDataPath);
23          }
24      }
25
26      [TestCleanup]
27      public void Teardown()
28      {
29          taskManager.Dispose();
30
31          if (File.Exists(testDataPath))
32          {
33              File.Delete(testDataPath);
34          }
35      }
36
37      [TestMethod]
38      public void TestAddTask_ShouldAddTaskToList()
39      {
40          var testTask = new Task
41          {
42              Title = "test_title",
43              Description = "test_description"
44          };
45
46          taskManager.AddTask(testTask);
47
48          Assert.AreEqual(1, taskManager.GetAllTasks().Count());
49          Assert.AreEqual("test_title", taskManager.GetAllTasks().First().Title);
50      }
51
52      [TestMethod]
53      public void TestFindTask_ShouldFindTask()
54      {
55          var testTask = new Task
56          {
57              Title = "test_title",
58              Description = "test_description"
59          };
60
61          taskManager.AddTask(testTask);
62
63          Assert.AreEqual("test_title", taskManager.GetTaskById(testTask.Id).Title);
64      }
65  }

```

(Рис.12 – Тесты)

```

KarakulovTests
[TestMethod]
public void TestGetAllTasks_ShouldGetAllTasks()
{
    var testTask1 = new Task
    {
        Title = "test_title1",
        Description = "test_description1"
    };

    var testTask2 = new Task
    {
        Title = "test_title2",
        Description = "test_description2"
    };

    taskManager.AddTask(testTask1);
    taskManager.AddTask(testTask2);

    Assert.AreEqual(2, taskManager.GetAllTasks().Count());
    Assert.AreEqual("test_title", taskManager.GetAllTasks().First().Title);
}

[TestMethod]
public void TestUpdateTask_ShouldUpdateExistingTask()
{
    var testTask = new Task
    {
        Title = "original_title",
        Description = "original_description"
    };

    taskManager.AddTask(testTask);

    testTask.Title = "updated_title";
    testTask.Description = "updated_description";

    taskManager.UpdateTask(testTask);

    var updatedTask = taskManager.GetTaskById(testTask.Id);
    Assert.IsNotNull(updatedTask);
    Assert.AreEqual("updated_title", updatedTask.Title);
    Assert.AreEqual("updated_description", updatedTask.Description);
}

[TestMethod]
public void TestRemoveTask_ShouldRemoveTaskFromList()
{
    var testTask = new Task
    {
        Title = "task_to_delete"
    };

    taskManager.AddTask(testTask);
    Assert.AreEqual(1, taskManager.GetAllTasks().Count());

    taskManager.RemoveTask(testTask);

    Assert.AreEqual(0, taskManager.GetAllTasks().Count());
}

[TestMethod]
public void TestSaveAndLoadFromFile_ShouldPersistTasks()
{
    var testTask1 = new Task { Title = "test_task1", Description = "test_desc1" };
    var testTask2 = new Task { Title = "test_task2", Description = "test_desc2" };

    taskManager.AddTask(testTask1);
    taskManager.AddTask(testTask2);

    taskManager.SaveToFile(testDataPath);

    var newTaskManager = new TaskManager();
    newTaskManager.LoadFromFile(testDataPath);
}

```

(Рис.13 – Тесты)

Визуализатор тестов

Запуск тестов завершен: тестов запущено в 992 мс: 6 (пройдено: 6, не пройдено: 0, пропущено: 0).

Тестирование	Длительность	Признаки	Сообщение об ошибке
✓ KarakulovTests (6)	438 мс		
✓ KarakulovTests (6)	438 мс		
✓ TestAddTask_ShouldAddTaskTo...	19 мс		
✓ TestFindTask_ShouldFindTask	< 1 мс		
✓ TestGetAllTasks_ShouldGetAllTa...	< 1 мс		
✓ TestRemoveTask_ShouldRemov...	1 мс		
✓ TestSaveAndLoadFromFile_Sho...	417 мс		
✓ TestUpdateTask_ShouldUpdate...	1 мс		

Предупреждения: 2 Ошибок: 0

Выполнить Отладка Профилировать

Сводка по группе

KarakulovTests

Тесты в группе: 6

Общая длительность: 438 мс

Результаты

6 Пройден

(Рис.14 – Запуск тестов)

РАЗДЕЛ 5. ИНСПЕКТИРОВАНИЕ КОМПОНЕНТ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ НА ПРЕДМЕТ СООТВЕТСТВИЯ СТАНДАРТАМ КОДИРОВАНИЯ

5.1. Процесс инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Инспектирование кода представляет собой процесс систематической проверки исходного кода на предмет соответствия установленным стандартам и рекомендациям по написанию кода. Основная цель инспектирования — обеспечение высокого качества кода, его читаемости, поддерживаемости и согласованности в рамках проекта. В ходе разработки приложения «KarakulovLib» инспектирование проводилось на всех этапах создания программного обеспечения.

Виды инспекции

В рамках проекта применялись следующие виды инспекции кода:

1. Статический анализ кода — автоматизированная проверка исходного кода без его фактического выполнения. Статический анализ позволяет выявлять потенциальные проблемы на ранних стадиях разработки: нарушения стиля кодирования, неиспользуемые переменные, избыточные конструкции, потенциальные ошибки и уязвимости. Данный вид инспекции выполнялся с использованием встроенных и внешних анализаторов, работающих на платформе .NET Compiler Platform (Roslyn).

2. Динамический анализ кода — проверка кода в процессе его выполнения. Данный вид инспекции проводился в ходе отладки и тестирования приложения, когда отслеживалось поведение программы при различных входных данных и сценариях использования. Динамический анализ позволяет выявлять ошибки, которые невозможно обнаружить статическими методами (например, логические ошибки, проблемы с производительностью, утечки памяти).

3. Ручное рецензирование кода (Code Review) – проверка кода разработчиком или группой разработчиков с целью выявления дефектов, оценки качества и соответствия стандартам. В рамках учебного проекта ручное рецензирование выполнялось автором при самостоятельном пересмотре написанного кода, а также в процессе консультаций с преподавателем. Особое внимание уделялось соблюдению принципов ООП, корректности использования паттерна MVVM, а также качеству комментариев и документации.

4. Формальная инспекция – структурированный процесс проверки кода с чётко определёнными ролями (автор, инспектор, модератор) и этапами (подготовка, обзор, исправление, повторная проверка). Ввиду небольшого объёма учебного проекта формальная инспекция применялась в упрощённом виде — с фиксацией замечаний в процессе написания и отладки кода.

Актуальные стандарты кодирования

При разработке приложения использовались следующие стандарты и рекомендации по кодированию:

1. Стандарты кодирования .NET и C# (Microsoft) – базовый набор правил и рекомендаций, используемых командой Microsoft для разработки примеров и документации. Данные стандарты были приняты из руководств по стилю кодирования .NET Runtime, C# Coding Style и C# compiler (Roslyn). Основные принципы включают:

- Правильность: код должен быть устойчивым и корректным.
- Обучаемость: код должен быть понятным и демонстрировать современные возможности языка.
- Согласованность: все фрагменты кода должны соответствовать единому стилю.
- Актуальность: использование современных языковых конструкций и версий C#.

2. Стандарты именования идентификаторов – в соответствии с рекомендациями Microsoft:

- Имена классов, методов и свойств должны начинаться с заглавной буквы (PascalCase).
- Имена параметров и локальных переменных должны начинаться со строчной буквы (camelCase).
- Интерфейсы должны начинаться с заглавной буквы I (например, ITaskRepository).
- Имена частных полей рекомендуется начинать с символа подчёркивания (_).
- Имена перечислений должны быть существительными в единственном числе (для не флаговых) или во множественном числе (для флаговых).

3. Правила оформления кода – включают использование отступов, пробелов, расположение фигурных скобок, длину строк и другие аспекты форматирования, обеспечивающие читаемость кода.

4. Правила качества кода – рекомендации по обработке исключений (перехватывать только те исключения, которые могут быть корректно обработаны; избегать перехвата общих исключений), использованию современных языковых конструкций, избеганию устаревших конструкций.

5. Принципы объектно-ориентированного программирования – инкапсуляция, наследование, полиморфизм и абстракция, реализованные в классах библиотеки KarakulovLib.

Программы, при помощи которых осуществляется инспекция

Для проведения инспекции кода на соответствие стандартам использовались следующие программные средства:

1. Встроенные анализаторы кода Visual Studio (Roslyn Analyzers) – .NET Compiler Platform (Roslyn) анализаторы встроены в Visual Studio Community 2022 и автоматически проверяют код на стиль, качество, поддерживаемость, дизайн и другие аспекты. Анализаторы работают в режиме реального времени на всех открытых файлах и отображают

нарушения в виде предупреждений и рекомендаций непосредственно в редакторе код

Результаты инспектирования

В ходе инспектирования кода приложения «Task Manager WPF» были выявлены и устранены следующие типичные нарушения:

- Несоответствие стандартам именования (отдельные поля и локальные переменные названы с заглавной буквы).
- Отсутствие XML-комментариев для публичных методов и классов.
- Использование устаревших конструкций (заменены на современные эквиваленты).
- Наличие неиспользуемых переменных и методов (удалены).
- Нарушения форматирования (отступы, лишние пробелы) — исправлены автоматически с помощью функции форматирования Visual Studio.

В результате инспектирования код проекта приведён к единому стилю, соответствует стандартам кодирования .NET и C#, а также принципам ООП. Все модульные тесты успешно проходят, что подтверждает корректность и надёжность разработанного программного обеспечения.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики с 8 по 28 июня 2026 г. были закреплены теоретические знания и расширены профессиональные умения в области разработки программного обеспечения на платформе .NET с использованием языка C#.

По итогам практики получены следующие результаты:

1. Разработано полнофункциональное приложение, реализующее управление списком задач с возможностью добавления, просмотра, фильтрации, поиска, редактирования и удаления записей, а также сохранения и загрузки данных в формате JSON. Приложение построено по архитектурному паттерну MVVM с разделением бизнес-логики (библиотека классов KarakulovLib) и пользовательского интерфейса (WPF-клиент).

2. Выполнен анализ предметной области: описаны бизнес-процессы в состояниях «AS IS» и «TO BE», проведён их сравнительный анализ, показаны преимущества автоматизации, сформулированы требования к программному обеспечению. Составлено техническое задание в соответствии с требованиями ГОСТ 19.201-78 и ГОСТ 34.602-89, включающее структуру, содержание и порядок документирования требований.

3. Создан набор модульных тестов (unit-тестов) покрывающих ключевые методы библиотеки классов (добавление, поиск, получение всех задач, обновление, удаление, сохранение и загрузка из файла). Все тесты успешно выполнены, что подтверждает корректность и стабильность работы программных модулей.

4. Проведена отладка приложения с применением специализированных средств среды Visual Studio Community 2022 (отладчик, точки останова, окна контроля значений, трассировка привязок WPF, диагностические инструменты). Выявлены и устранены ошибки в логике поиска, фильтрации, сериализации, обновления интерфейса и обработке

исключений, что обеспечило надёжную работу всех функциональных возможностей.

5. Осуществлена инспекция кода на соответствие актуальным стандартам кодирования .NET и C# (правила именования, оформления, качества и дизайна). Для автоматизированной проверки использовались встроенные анализаторы Visual Studio (Roslyn), а также ручное рецензирование. Все выявленные нарушения стиля и потенциальные проблемы были исправлены, код приведён к единому стандарту.

По окончании практики был получен практический опыт разработки требований к программным модулям на основе анализа проектной и технической документации на предмет взаимодействия компонент; интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. Единая система программной документации (ЕСПД). Техническое задание на разработку программы или программного изделия. – М.: Стандартинформ, 1978.
2. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М.: Стандартинформ, 1989.
3. ГОСТ 19.101-77. Единая система программной документации (ЕСПД). Виды программных документов. – М.: Стандартинформ, 1977.
4. Microsoft Corporation. Общие соглашения о кодировании на С# и правила именования идентификаторов. – URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/fundamentals/coding-style/identifier-names> (дата обращения: 20.06.2026).
5. Microsoft Corporation. Создание WPF-приложения на С# с использованием паттерна MVVM. Документация по Visual Studio. – URL: <https://learn.microsoft.com/ru-ru/visualstudio/get-started/csharp/tutorial-wpf> (дата обращения: 20.06.2026).
6. Microsoft Corporation. Модульное тестирование С# в .NET с использованием MSTest. – URL: <https://learn.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-csharp-with-mstest> (дата обращения: 20.06.2026).
7. Microsoft Corporation. Отладка кода на С# с использованием отладчика Visual Studio. – URL: <https://learn.microsoft.com/ru-ru/visualstudio/get-started/csharp/tutorial-debugger> (дата обращения: 20.06.2026).

Требование к написанию программного модуля

Задание.

Цель:

Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования: Разработать приложение для управления задачами с возможностью:

1.1 Добавления новой задачи (название, описание, приоритет, срок выполнения, статус).

1.2 Просмотра списка всех задач.

1.3 Фильтрации задач по статусу (например, "Новая", "В процессе", "Завершена").

1.4 Поиска задач по названию или описанию.

1.5 Редактирования существующих задач.

1.6 Удаления задач.

1.7 Сохранения и загрузки задач в/из файла (JSON или XML).

1.8 Основная логика приложения должна находиться в динамической библиотеке, а пользовательский интерфейс должен представлять из себя WPF-приложение.

1.9 Для классов в библиотеке необходимо написать модульные тесты.

2. Дополнительные задания (по желанию)

2.1 Реализовать сортировку задач по приоритету или сроку выполнения.

2.2 Добавить возможность пометки задачи как "Важная" (выводить звёздочкой или цветом в консоли).

2.3 Реализовать простую статистику (например, сколько задач завершено, сколько просрочено).

Сопроводительная записка должна содержать:

- титульный лист,
- оглавление,
- диаграмму классов,
- подробное описание действий по созданию ПО (с иллюстрациями).

Исходные коды приложения и сопроводительную записку выложить на GitHub и ссылку на репозиторий прислать мне на почту.